

Recuperacion de Bundle a Producto en Moda: Metodologia CRISP-DM y Documentacion Tecnica

Equipo goofytex
Informe Tecnico Interno

Abstract—Este documento describe el sistema de retrieval bundle→product de goofytex usando metodologia CRISP-DM. Se cubren datos, preprocesamiento, modelado, implementacion e inferencia del repositorio, incluyendo familias de embeddings probadas (QRLite, OpenCLIP Marqo Fashion SigLIP y SigLIP), deteccion de prendas basada en YOLO y variantes de pipeline. El foco es tecnico y reproducible: se documentan componentes, flujos y decisiones de ingenieria sin reportar resultados cuantitativos.

Index Terms—retrieval visual, CRISP-DM, OpenCLIP, SigLIP, YOLO, IA en moda

I. INTRODUCCION

En goofytex, la tarea se formula como retrieval visual multietiqueta: dada una imagen de bundle, el sistema debe recuperar y ordenar los `product_asset_id` mas probables, con un maximo de 15 predicciones por bundle.

El repositorio implementa un stack modular orientado a experimentacion rapida y despliegue reproducible:

- 1) Preparacion de datos e imagenes (`download.sh`, `preprocess_data.py`, `offline_augment.py`).
- 2) Entrenamiento de retrieval multimodal (`python -m src.train`, backend `src/models/retrieval_openclip.py`).
- 3) Inferencia y exportacion de submission (`src.infer`, `src.new_infer`, `src.infer_phase1`, `src.infer_top1`, `src.infer_openclip`).

La documentacion adopta CRISP-DM como marco de trabajo para organizar decisiones tecnicas, trazabilidad y fases de iteracion. El objetivo de este paper es consolidar la metodologia y las tecnologias usadas en el codigo, sin incluir resultados de rendimiento.

II. DATOS Y PREPROCESAMIENTO

A. Datasets de origen

Las fuentes de datos principales se resumen en la Tabla I.

B. Pipeline de preprocesamiento

`preprocess_data.py` implementa validacion de esquema, chequeos de integridad referencial, split train/val por `bundle_asset_id`, descarga opcional de imagenes y generacion de manifestos. Los artefactos principales son:

- `data/manifests/train_manifest.jsonl`
- `data/manifests/val_manifest.jsonl`
- `data/labels.json`, `data/label2idx.json`, `data/stats.json`

Archivo	Proposito
<code>bundles_dataset.csv</code>	Catalogo de bundles y URLs de imagen.
<code>product_dataset.csv</code>	Catalogo de productos, URLs y descripcion textual.
<code>bundles_product_match_train.csv</code>	Entrenamiento bundle-producto para entrenamiento.
<code>bundles_product_match_test.csv</code>	Entrenamiento de test para predecir <code>product_asset_id</code> .

TABLE I

DATASETS USADOS POR ENTRENAMIENTO E INFERENCIA.

C. Aumento offline

`offline_augment.py` genera vistas sinteticas para bundles y productos con semillas deterministas por `asset_id+indice`. El objetivo es ampliar variabilidad visual sin romper trazabilidad de datos.

Artefactos tipicos del proceso:

- `bundles_aug/*.jpg`, `bundles_aug_manifest.jsonl`
- `products_aug/*.jpg`, `products_aug_manifest.jsonl`

III. METODOLOGIA (CRISP-DM)

A. Marco CRISP-DM aplicado

La metodologia del proyecto sigue CRISP-DM y se adapta directamente a los scripts del repositorio (Tabla II).

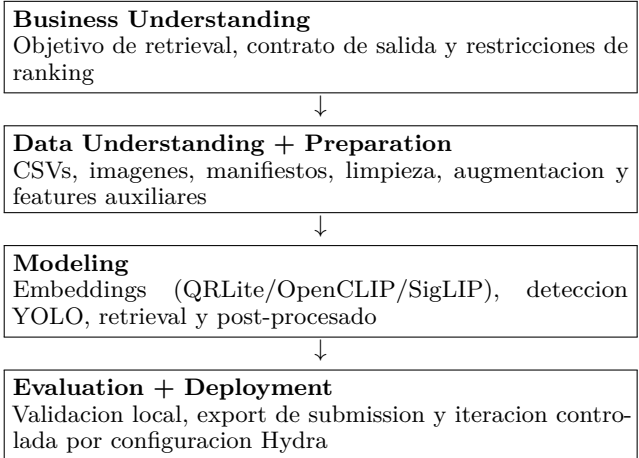


Fig. 1. Diagrama de alto nivel del flujo CRISP-DM en el proyecto.

Fase DM	CRISP-	Aplicacion en goofytex
Business Understanding		Definicion del objetivo bundle→product y del formato de entrega (maximo 15 predicciones por bundle).
Data Understanding		Inspeccion de CSVs, mapeo de imagenes y analisis de metadata (scripts en <code>preprocess_data.py</code> y <code>src/utlis</code>).
Data Preparation		Limpieza de etiquetas, manifests train/val, descarga/validacion de imagenes y aumentacion offline determinista.
Modeling		Entrenamiento retrieval multimodal con OpenCLIP, pruebas de familias de embedding alternativas y deteccion de regiones con YOLO.
Evaluation		Evaluacion local de ranking en validacion y chequeos funcionales del pipeline por script/variante.
Deployment		Generacion de <code>submission.csv</code> , configuracion con Hydra y ejecucion reproducible por CLI.

TABLE II

MAPEO CRISP-DM A COMPONENTES REALES DEL REPOSITORIO.

B. Modelos de embeddings probados

Durante la iteracion metodologica se trabajaron varias familias de embedding visual/multimodal. Entre ellas, la linea CLIP/SigLIP [1], [2] estructura el backend principal de representacion. La Tabla III resume su papel.

Familia de modelo	Estado	Uso en el flujo
QRLite	Probado	Alternativa ligera para prototipos de embeddings y comparacion de coste/inferencia.
OpenCLIP Marqo Fashion SigLIP	Principal	Backend actual de entrenamiento e inferencia multimodal (scripts <code>train</code> , <code>infer</code> y <code>new_infer</code>).
SigLIP	Probado	Familia contrastiva adicional usada en pruebas de embeddings y zero-shot de categorias.
Torchvision CNNs (ResNet18/ResNet50/EfficientNet-B0)	Baseline	Encoders visuales preentrenados para baseline y comparativas de retrieval.

TABLE III

FAMILIAS DE EMBEDDING CONSIDERADAS EN LA METODOLOGIA.

C. Deteccion de prendas con YOLO

Para capturar objetos dentro del bundle se integra deteccion por regiones. El repositorio usa por defecto `kesimeg/yolov8n-clothing-detection` mediante `ultralyticsplus`, y la metodologia contempla pruebas con varias configuraciones/checkpoints YOLO [3] para ajustar granularidad de crops.

Componente YOLO	Implementacion	Funcion en pipeline
YOLO clothing (familia YOLOv8)	<code>src/detection.py</code>	Deteccion de cajas por bundle y cache JSON reutilizable.
Variantes de umbral (conf/IoU)	Hydra config	Control de precision/cobertura de cajas antes del retrieval por crop.
Fallback imagen completa	<code>src.infer*</code>	Garantiza inferencia cuando no hay detecciones validas.

TABLE IV

USO METODOLOGICO DE YOLO EN LA FASE DE MODELADO/INFERENCIA.

D. Pipeline tecnico end-to-end

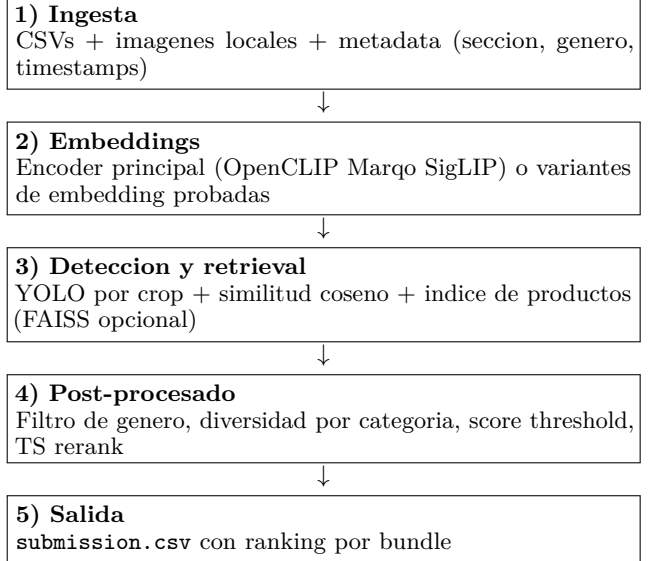


Fig. 2. Pipeline bundle→product implementado en los scripts de inferencia.

E. Recuperacion y scoring

La similitud principal se calcula por coseno sobre embeddings normalizados:

$$\text{sim}(\mathbf{q}, \mathbf{P}) = \mathbf{qP}^T \quad (1)$$

En entrenamiento (`src/models/retrieval_openclip.py`) se usa perdida contrastiva bidireccional bundle↔product, con scheduler coseno, warmup, AMP opcional y mineria de negativos duros configurable.

F. Variantes de inferencia implementadas

- `src.infer`: pipeline principal (OpenCLIP + YOLO + filtros).
- `src.new_infer`: retrieval per-crop con TTA y FAISS opcional.
- `src.infer_phase1`: filtrado por seccion + zero-shot por categoria.

- `src.infer_top1`: estrategia compacta top-1 por crop.
- `src.infer_openclip`: script legacy por argparse.

IV. DETALLES DE IMPLEMENTACION

A. Configuración y entrypoints

El proyecto usa Hydra [4] con dataclasses tipadas (`src/config.py`). Entrypoints operativos:

- `python -m src.train`: entrenamiento OpenCLIP.
- `python -m src.infer`: inferencia principal con salida en directorio Hydra.
- `python -m src.new_infer`: inferencia per-crop con TTA y FAISS opcional.
- `python -m src.infer_phase1`: inferencia por seccion + zero-shot.
- `python -m src.infer_top1`: inferencia top-1 por crop.
- `python -m src.infer_openclip`: inferencia simple por CLI.

B. Mapa de módulos

Modulo	Responsabilidad
<code>train.py</code>	Bootstrap Hydra y despacho de backend de entrenamiento.
<code>retrieval_openclip.py</code>	Loop train/val, contrastive loss, minería de negativos, checkpoints y métricas JSONL.
<code>infer.py</code>	Inferencia principal con detección opcional y post-procesado configurable.
<code>new_infer.py</code>	Variante per-crop + TTA + índice FAISS opcional + TS rerank.
<code>infer_phase1.py</code>	Variante por seccion y zero-shot de categoría por crop.
<code>infer_top1.py</code>	Variante top-1 por crop para salidas compactas.
<code>infer_openclip.py</code>	Script legacy de inferencia con argumentos CLI.
<code>detection.py</code>	Wrapper de YOLO clothing, NMS y cache de cajas.
<code>feature_extractors.py</code>	Baselines de embeddings con torchvision (ResNet/EfficientNet).
<code>metrics.py</code>	Métricas Hit@K y Recall@K.
<code>add_gender.py</code>	Enriquecimiento de género para productos.
<code>check_link_timestamps.py</code>	Análisis de coherencia temporal bundle-producto.

TABLE V

MODULOS PRINCIPALES Y RESPONSABILIDADES TECNICAS.

C. Stack tecnologico observado en el repositorio

V. PLAN EXPERIMENTAL (SIN RESULTADOS)

A. Objetivo del plan

Esta seccion define el protocolo de experimentacion para comparar variantes de pipeline sin reportar metricas finales. El foco es describir *que* se prueba y *como* se ejecuta cada prueba.

Tecnologia	Tipo	Uso en el proyecto
Python	Lenguaje	Implementacion de pipelines de datos, entrenamiento e inferencia.
PyTorch	Deep learning	Entrenamiento/inferencia, autodiff, AMP y DataParallel.
Torchvision	Model zoo/preprocess	Baselines de embeddings (ResNet18/50, EfficientNet-B0).
OpenCLIP (<code>open_clip_torch</code>)	Multimodal encoder	Backend principal de embeddings y retrieval (Marqo Fashion SigLIP).
SigLIP	Familia de modelo	Base conceptual de los encoders contrastivos usados/probados.
Hydra (<code>hydra-core</code>)	Configuracion	Gestion de parametros, rutas y salidas reproducibles por run.
UltralyticsPlus (YOLO)	Deteccion de objetos	Deteccion de prendas y recorte por regiones de bundle.
FAISS (opcional)	ANN index	Aceleracion de busqueda de vecinos en <code>src.new_infer.py</code> .
NumPy / Pandas	Data processing	Manipulacion tabular y operaciones vectoriales de retrieval.
Pillow	Vision I/O	Lectura y transformacion de imagenes.
tqdm	Observabilidad	Progreso de etapas largas (encoding, deteccion, retrieval).
requests	I/O red	Descarga y validacion de recursos en preprocesamiento.
LaTeX (IEEEtran)	Documentacion tecnica	Paper de metodologia, arquitectura y reproducibilidad.

TABLE VI

TECNOLOGIAS UTILIZADAS Y LOCALIZADAS EN EL CODIGO/DOCUMENTACION.

B. Matriz de experimentos propuesta

C. Protocolo operativo

- 1) Fijar semilla y configurar run con Hydra.
- 2) Codificar índice de productos para la variante seleccionada.
- 3) Ejecutar inferencia sobre split de validacion local controlado por `infer.val_ratio`.
- 4) Exportar artefactos de run para trazabilidad y comparacion posterior.

D. Artefactos esperados por run

- `metrics.jsonl`, `best.pt`, `epoch_X.pt` en entrenamiento.
- `test_submission*.csv` y `val_metrics*.json` en inferencia.
- Caches de deteccion en JSON bajo `data/yolo_detections` u otra ruta configurada.

Eje	Variantes	Scripts / Config
Embeddings	QRLite, OpenCLIP Marqo SigLIP, SigLIP, torchvision baselines	train, infer, feature_extractors
Deteccion	Full-image only vs YOLO por crops, variando conf/IoU y max boxes	detection.py, parametros bbox_*
Retrieval in- dex	Dot-product directo vs FAISS (opcional)	src.new_infer.py
Post- procesado	Gender filter, dedup por categoria, score threshold, TS rerank	infer.* en Hydra
Estrategia fi- nal	infer, new_infer, infer_phase1, infer_top1	Entrypoints de infer- encia

TABLE VII

MATRIZ DE COMPARATIVAS PLANIFICADAS EN EL REPOSITORIO.

VI. REPRODUCIBILIDAD

A. Entorno

Dependencias base en `requirements.txt`: `pandas`, `numpy`, `pillow`, `tqdm`, `requests`, `hydra-core`, `torch`, `torchvision`.

Dependencias adicionales para pipelines avanzados:

- `open_clip_torch` (backend OpenCLIP).
- `ultralyticsplus` (deteccion YOLO clothing).
- `faiss` (opcional en `src.new_infer.py`).

B. Determinismo

El codigo fija semillas de `random`, `numpy` y `torch`. `offline_augment.py` usa semillas estables por hash de `asset_id`+indice para reproducir exactamente las mismas vistas sinteticas.

C. Configuracion Hydra

La configuracion esta centralizada en:

- `config/config.yaml`: hiperparametros de train/infer.
- `config/files/file.yaml`: rutas de datos, imagenes y caches.

Cada run genera un directorio de salida versionado por fecha/hora, lo que permite comparar variantes sin gestionar rutas manualmente.

D. Contrato de entrega

El CSV de submission debe contener:

- `bundle_asset_id`
- `product_asset_id`

con un maximo de 15 filas por bundle y orden por confianza del modelo.

VII. LIMITACIONES Y RIESGOS

- 1) **Supervision visual limitada por producto:** muchos IDs tienen pocas vistas, lo que complica robustez ante cambios de pose y oclusion.

- 2) **Dependencia de modelos externos:** OpenCLIP y YOLO requieren pesos externos y versionado consistente de entorno.
- 3) **Coste computacional:** deteccion por crops y rerankings incrementan latencia frente a pipelines full-image.
- 4) **Dependencias opcionales no unificadas:** FAISS y Ultralytics se instalan aparte de `requirements.txt`.
- 5) **Cobertura de tests limitada:** no hay una suite extensa de tests automatizados para todas las variantes de inferencia.

VIII. CONCLUSIONES

El repositorio goofytex implementa un stack completo para retrieval bundle→product con enfoque CRISP-DM: preparacion de datos, modelado multimodal, deteccion por regiones, inferencia y export de entregas.

La documentacion consolidada describe de forma operativa las familias de modelos probadas (QRLite, OpenCLIP Marqo Fashion SigLIP y SigLIP), el uso de multiples configuraciones YOLO y el conjunto de tecnologias empleadas en codigo y ejecucion.

Este documento queda como especificacion tecnica para continuar iterando sobre el pipeline con criterios de trazabilidad y reproducibilidad.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the International Conference on Machine Learning*, 2021.
- [2] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *arXiv preprint arXiv:2303.15343*, 2023.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” GitHub repository, 2023.
- [4] O. Yadan, “Hydra: A framework for elegantly configuring complex applications,” GitHub repository, 2019.

APPENDIX

APENDICE: COMANDOS REPRODUCIBLES

A. Instalacion

```
pip install -r requirements.txt
pip install open_clip_torch ultralyticsplus
# opcional: pip install faiss-cpu
```

B. Descarga de imagenes

```
bash download.sh
```

C. Preprocesamiento

```
python preprocess_data.py \
  --skip_download \
  --out_dir data \
  --val_ratio 0.1 \
  --seed 42
```

D. Generacion de metadata de genero

```
python src/utis/add_gender.py
```

E. Entrenamiento (Hydra)

```
python -m src.train
```

F. Inferencia principal

```
python -m src.infer infer.checkpoint_path=<ruta_a_best.pt>
```

G. Inferencia v8

```
python -m src.new_infer infer.checkpoint_path=<ruta_a_best.pt>
```

H. Inferencia Phase 1

```
python -m src.infer_phase1 infer.checkpoint_path=<ruta_a_best.pt>
```

I. Inferencia Top-1

```
python -m src.infer_top1 infer.checkpoint_path=<ruta_a_best.pt>
```

J. Script legacy OpenCLIP

```
python -m src.infer_openclip \  
    --checkpoint outputs/retrieval_openclip/best.pt \  
    --submission-out outputs/retrieval_openclip/submission.csv
```

K. Reporte de timestamps

```
python src/utils/check_link_timestamps.py \  
    --out-csv outputs/ts_date_alignment_report.csv \  
    --timezone UTC
```